BASIC NETWORK SNIFFER USING PYTHON

1. Objective

The objective of this project is to build a basic Python network sniffer that captures IPv4 packets visible to the selected network interface and displays useful metadata. The project helps demonstrate packet structure, IP addressing, transport protocols, and the role of packet analysis in network troubleshooting.

2. Tools and Requirements

- Python 3
- Scapy Python library
- Npcap on Windows, where required for packet capture
- Visual Studio Code or another Python editor

3. Methodology

1. Import Scapy functions for packet capture and protocol-layer inspection.
2. Capture up to 30 IPv4 packets using sniff(), with store=False to avoid retaining a packet list.
3. Inspect the IP layer to read source and destination IPv4 addresses.
4. Identify TCP, UDP, and ICMP packets and display transport ports when applicable.
5. Display the payload length and a short packet summary without printing payload contents.

4. Information Displayed

Field	Description
Timestamp	Local time when the packet is processed by the program.
Source IP	IPv4 address shown as the packet source.
Destination IP	IPv4 address shown as the packet destination.
Protocol	TCP, UDP, ICMP, or the numeric IP protocol identifier.
Ports	TCP/UDP source and destination ports, where applicable.
Payload size	Bytes in the inspected transport payload section; not necessarily a complete application message.
Packet summary	A concise Scapy summary of the packet layers.

5. How to Run

- Install Python 3 and Scapy: python -m pip install scapy
- On Windows, install Npcap from https://npcap.com/ if needed.
- Open a terminal in the folder containing sniffer.py.
- Run: python sniffer.py
- Generate ordinary network activity on your own device and observe the output. Capture only traffic you are authorized to inspect.

6. Sample Output (Illustrative)

----------\nTime: 20:18:02\nSource IP: 192.168.1.10\nDestination IP: 142.250.183.14\nProtocol: TCP\nSource Port: 52140\nDestination Port: 443\nPayload Size: 517 bytes\nPacket Summary: Ether / IP / TCP / Raw

Note: The values above are illustrative examples, not actual captured results. Replace this section with your own output screenshot or observations after running the program.

7. Observations and Learning

The program demonstrates how packet metadata can be inspected to identify IP endpoints, transport protocols, ports, and payload length. TCP provides a reliable ordered byte stream, UDP provides datagram delivery without TCP's delivery guarantees, and ICMP supports network control and diagnostics. A packet sniffer can only capture traffic visible to its interface and capture permissions.

8. Limitations

- This version focuses on IPv4 and common TCP, UDP, and ICMP metadata.
- It does not reassemble TCP streams or decode application-level messages.
- Payload contents are not printed; HTTPS application data is normally encrypted in transit.
- Capture availability depends on the operating system, network interface, permissions, and Npcap/libpcap support.
- The sample output is illustrative until replaced with evidence from an actual run.

9. Ethical and Legal Considerations

Packet capture should be performed only on the user's own device or a network for which explicit authorization has been granted. Avoid collecting, storing, or sharing other people's private traffic, credentials, or sensitive information. Redact sensitive details in screenshots.

10. Conclusion

This project provides a starting point for learning packet capture and basic network protocol analysis using Python and Scapy. After running the program, actual observations and a screenshot should be added to document successful testing.

Student name: Shivam Kumar Choudhary

Date: 12-10-2026